

구조 패턴

구조패턴이란?

클래스나 객체를 조합해 더 큰 구조를 만드는 패턴

구조패턴의 특징

서로 독립적으로 개발한 클래스 라이브러리를 하나인 것처럼 사용 가능

- 여러 인터페이스를 합성하여 서로 다른 인터페이스들의 통일된 추상을 제공

☞ 어댑터 패턴(Adapter Pattern) ★

코드를 짜다 보면 이미 잘 만들어진 클래스가 있는데, 그 클래스의 **인터페이스(반환 타입이나 메소드명)**가 내 코드와 맞지 않아서 바로 쓸 수 없는 경우가 생깁니다. 이때 사용하는 것이 **어댑터 패턴**입니다.

"호환되지 않는 인터페이스를 가진 클래스들이 함께 동작할 수 있도록, 클라이언트의 요구 타입과 반환 타입을 적절히 가공하여 연결해주는 패턴"

가장 쉬운 예시는 *****해외여행용 변환 플러그(돼지코)*****입니다.

- 한국 기기(Client):** 220V 둥근 모양을 원함.
- 미국 콘센트(Adaptee):** 110V 납작한 모양을 제공함.
- 어댑터(Adapter):** 110V를 받아서 220V 모양으로 바꿔줌으로써, 기기를 개조하지 않고도 전기를 쓸 수 있게 해줌.
- 예제코드

```
// long형 변수를 twoTime과 half메소드를 사용해서 계산하고싶음
//기존에 존재하는 코드//
public class Math {
    public static double twoTime(double num) {
        return num * 2;
    }

    public static double half(double num) {
        return num / 2;
    }
}
// double형이라 타입이 일치하지 않음 - 사용불가

//어댑터 만들기//
public interface Adapter {
    public long twiceOf(long l);
    public long halfOf(long l);
}
```

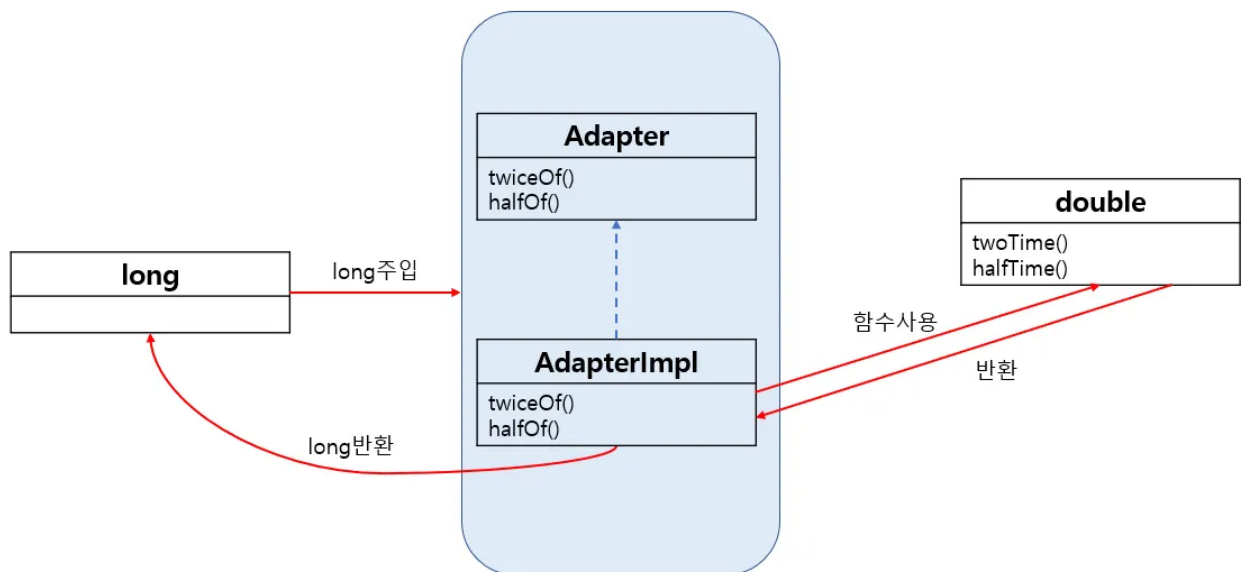
```

public class AdapterImpl implements Adapter {
    @Override
    public long twiceOf(long l) {
        return (long) Math.twoTime((double) l);
    }
    @Override
    public long halfOf(long l) {
        return (long) Math.half((double) l);
    }
}

public class main {
    public static void main(String[] args) {
        Adapter adapter = new AdapterImpl();
        System.out.println(adapter.twiceOf(100l)); //50
        System.out.println(adapter.halfOf(50l)); //25
    }
}

```

- 예제코드 UML



- 장점

1. 생산성 향상과 OCP 준수 (재사용성)

- 상황:** 기능은 똑같은데 메소드 이름만 다른 기존 코드가 있다고 칩시다.
- 해결:** 기존 코드를 뜯어고치는 것(위험함)이 아니라, 어댑터를 끼워서 내 입맛에 맞게 변환만 하면 됩니다.
- 결과:** 기존 코드를 재활용하므로 생산성이 높아지고, 기존 클래스를 수정하지 않으면서 새로운 인터페이스를 확장할 수 있어 ****OCP(개방-폐쇄 원칙)****를 지킬 수 있습니다.

2. 역할 분리와 SRP 준수 (유지보수)

- 상황:** 데이터 포맷을 XML에서 JSON으로 바꿔야 한다면?

- **해결:** 비즈니스 로직 클래스에 변환 코드를 섞지 않고, '변환 작업'만 담당하는 어댑터 클래스를 따로 만듭니다.
- **결과:** 기존 클래스는 자기 일만 하고, 어댑터는 변환 일만 하므로 **SRP(단일 책임 원칙)**을 지킬 수 있어 코드가 깔끔해집니다.
- 단점

1. 클래스가 많아져 복잡해진다

- 단순한 기능인데도 어댑터 클래스를 따로 만들어야 하므로 코드 구조가 복잡해질 수 있습니다. 인터페이스 차이가 크지 않다면 그냥 코드를 수정하는 게 나을 수도 있습니다.

"외부 결제 API(카카오페이, 네이버페이 등)를 연동할 때 이 패턴을 자주 씁니다. 각 회사마다 API 사용법이 다른데, 우리 서버에서는 `pay()`라는 하나의 통일된 메소드로 쓰고 싶을 때, 중간에 **어댑터**를 두어 각 회사의 API를 우리 방식대로 변환해서 사용합니다."

브리지 패턴(Bridge Pattern)

하나의 클래스 계층구조가 너무 복잡해져서, 상속만으로는 확장이 감당이 안 될 때가 있습니다. 이때 **다리(Bridge)**를 놓아 두 개의 계층으로 나누는 패턴이 바로 브리지 패턴입니다.

"구현부(Implementation)에서 추상층(Abstraction)을 분리하여, 이 둘이 서로 독립적으로 변화하고 확장할 수 있도록 하는 패턴"

기능 클래스 계층과 구현 클래스 계층을 연결다리(Bridge)로 연결해두고, 서로 남남처럼 관리하겠다는 뜻

- **기능(Abstraction):** "무엇을 할 것인가?" (예: 리모컨의 버튼 - 켜기, 끄기)
- **구현(Implementation):** "어떻게 할 것인가?" (예: 실제 TV의 동작 - 삼성 TV 켜짐, LG TV 켜짐)

■ 언제 사용하나요? (Why?)

1. 클래스가 폭발적으로 늘어나는 것을 막고 싶을 때 (M x N 문제 해결)

- **상황:** 모양(원, 네모)과 색상(빨강, 파랑)이라는 두 가지 기준이 있다고 칩시다. 상속을 쓰면 빨간 원, 파란 원, 빨간 네모, 파란 네모 등 총 4개(2x2)의 클래스가 필요합니다. 만약 모양이 3개, 색상이 3개가 되면 9개(3x3)가 필요하겠죠.
- **해결:** 브리지 패턴을 쓰면 모양 클래스 3개 + 색상 클래스 3개 = 총 6개만 있으면 됩니다. 모양이 색상을 가지는(Has-a) 구조로 연결하기 때문입니다.

2. 기능과 구현을 독립적으로 확장하고 싶을 때

- 새로운 기능을 추가해도(예: 리모컨에 '음소거' 추가) 기존 TV 구현 코드를 건드릴 필요가 없고, 새로운 TV 브랜드가 추가되어도 리모컨 코드를 건드릴 필요가 없습니다.
- 예제 코드

```
////////추가 규칙 정의 - 구체클래스////////
public interface Color{
    public String fill();
}
```

```
////////Color인터페이스를 상속하여 추가규칙 정의////////
public class Blue implements Color{
    public String fill(){
        return "파란색";
    }
}

public class Black implements Color{
    public String fill(){
        return "검정색";
    }
}

////////구현하기 - 추상클래스////////
public abstract class Brush{
    Color color;

    Brush(Color color){
        this.color = color;
    }

    public abstract String draw();
}

////////상속을 통한 Brush의 기능을 수행할 구체 클래스 정의////////
public class Pencil extends Brush{
    public static final String type = "색연필";

    Pencil(Color color){
        super(color);
    }

    public String draw(){
        return color.fill() + " " + type;
    }
}

public class Pastel extends Brush{
    public static final String type = "파스텔";

    Pencil(Color color){
        super(color);
    }

    public String draw(){
        return color.fill() + " " + type;
    }
}

////테스트 코드////
public class BridgeTest{
    public static void main(String[] args){
        Brush redPencil = new Pencil(new Red());
    }
}
```

```

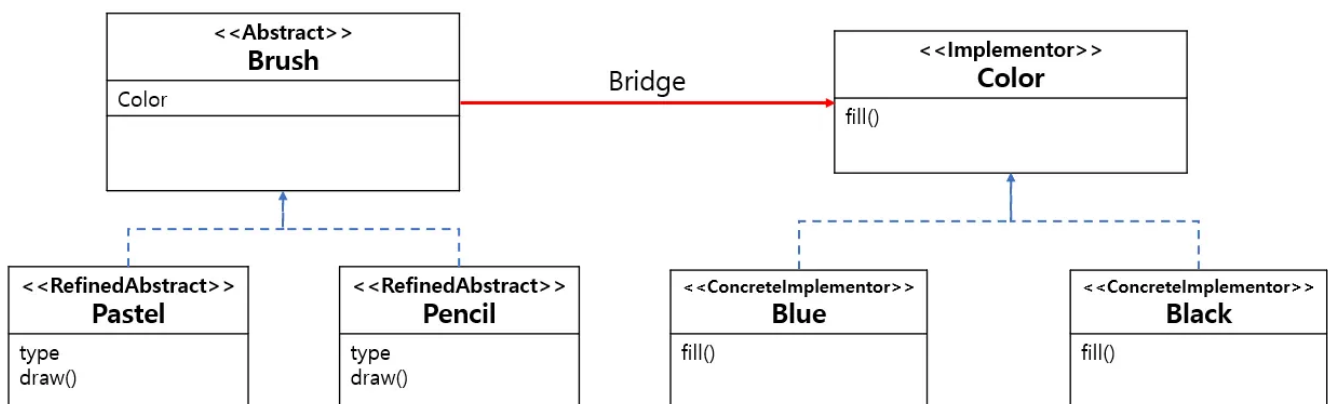
        Brush blackPastel = new Pastel(new Black());

        System.out.println(redPencil.draw());
        System.out.println(blackPastel.draw());
    }
}

///출력///
빨간색 색연필
검정색 파스텔
////////

```

- 예제코드 UML



■ 코드 기능 확장해보기

```

//1.노란색 추가하기
public class Yellow implements Color{
    public String fill(){
        return "노란색";
    }
}
///위 코드를 추가해도 이미 만들어둔 코드에 변화 x
//위 코드 추가로 [노란색 색연필, 노란색 파스텔] 생성가능

//2.볼펜 추가하기
public class BallPen extends Brush{
    public static final String type = "볼펜";

    BallPen(Color color){
        super(color);
    }

    public String draw() {
        return color.fill() + " " + type;
    }
}
///위 코드를 추가해도 이미 만들어둔 코드에 변화 x

```

```
// 위 코드 추가로 [빨간색 볼펜, 검정색 볼펜] 생성가능
```

- 장점

1. 확장성 향상 (독립적인 관리)

- 기능 계층과 구현 계층이 분리되어 있어, 어느 한쪽을 수정하거나 확장해도 다른 쪽에 영향을 주지 않습니다.

2. 구체적인 세부 정보를 클라이언트로부터 숨길 수 있다

- 클라이언트는 고수준의 기능(Abstraction)만 사용하면 되고, 내부적으로 어떤 구현체(Implementor)가 돌아가는지는 몰라도 됩니다.
- 단점

1. 코드가 복잡해진다

- 계층을 나누고 연결해야 하므로 클래스 개수가 늘어나고 구조가 복잡해질 수 있습니다. 간단한 기능에는 오버엔지니어링이 될 수 있습니다.
- **Abstraction (리모컨):** 사용자가 누르는 버튼(기능).
- **Implementor (TV):** 실제 전파를 받아 화면을 켜는 기계(구현).
- **Bridge:** 리모컨은 TV가 삼성인지 LG인지 상관없이, 연결된(Bridge) TV에 신호를 보냄. TV를 바꿔도 리모컨은 그대로 쓸 수 있음!
- 브리지 패턴의 예시: **JDBC**
 - 여러분이 코드에서 **Connection**을 사용할 때, 실제 뒤단에서 MySQL이 도는지 오라클이 도는지 신경 안 쓰죠? 자바는 "연결한다"는 기능만 정의하고, 실제 동작은 각 DB 회사가 만든 드라이버(구현체)에게 위임했기 때문입니다. 이 둘을 이어주는 것이 바로 **브리지 패턴**입니다.

🏠 컴포지트 패턴(Composite Pattern)

탐색기에서 '파일' 하나를 삭제할 때와 '폴더' 전체를 삭제할 때, 우리는 똑같이 **Delete** 키를 누릅니다. 컴퓨터 내부적으로는 처리 방식이 다르겠지만, 사용자 입장에서는 **파일이든 폴더든 똑같이 취급**하죠. 이것이 바로 **컴포지트 패턴**입니다.

"부분과 전체의 계층을 표현하기 위해 객체들을 트리 구조로 구성하고, 사용자가 단일 객체와 복합 객체(그룹)를 동일하게 다룰 수 있도록 하는 패턴"

■ 컴포지트(Composite)란?

- 하나 이상의 유사한 객체를 구성으로 설계된 객체
- 객체 그룹을 조작하는 것처럼 단일 객체를 조작가능

■ 언제 사용하나요? (Why?)

1. 데이터를 '트리 구조'로 표현해야 할 때

- 파일 시스템(디렉토리/파일), 조직도(부서/팀/직원), 메뉴 시스템(대분류/소분류/메뉴) 등 계층적인 구조를 가질 때 가장 적합합니다.

2. 전체와 부분을 구분 없이 다루고 싶을 때 (일관성)

- 클라이언트 코드가 "이게 폴더인가? 파일인가?"를 `if`문으로 일일이 따지지 않고, 그냥 `.print()`나 `.delete()`를 호출하면 알아서 동작하게 만들고 싶을 때 사용합니다.
- 예제코드

```
//컴포넌트정의//
public interface Unit {
    public void move();
}

//Leaf 정의//
public class SCV implements Unit{ //단일객체

    @Override
    public void move(){
        System.out.println("SCV 이동");
    }
}

public class Marine implements Unit{ //단일객체

    @Override
    public void move(){
        System.out.println("마린 이동");
    }
}

public class Tank implements Unit{ //단일객체

    @Override
    public void move(){
        System.out.println("탱크 이동");
    }
}

/////컴포지트 정의/////
public class Moving implements Unit{ //복합객체

    //컴포넌트를 담는 배열리스트 추가
    private List<Unit> units = new ArrayList<>();

    //군집 이동
    @Override
    public void move(){
        for(Unit u : units){ //배열리스트에 추가된 모든 컴포넌트 출력
            u.move();
        }
    }
}
```

```

//컴포넌트 배열리스트에 추가
public void add(Unit u){
    this.units.add(u);
}

//컴포넌트 배열리스트에서 삭제
public void kill(Unit u){
    this.units.remove(u);
}

//배열리스트의 모든 컴포넌트 삭제
public void allKill() {
    this.units.clear();
}
}

public class CompositePatternTest{
    public static void main(String[] args){
        Unit marine = new Marine();
        Unit SCV = new SCV();
        Unit tank = new Tank();

        Moving mv = new Moving();
        mv.add(marine);
        mv.add(SCV);
        mv.add(tank);

        mv.move();

        System.out.println("\n\n테스트 코드에서 새로만든 units배열출력");
        List<Unit> = units = new ArrayList<>();
        units.add(mv); //Moving도 하나의 Unit객체로 취급
        units.add(new Tank());
        units.add(new SCV());

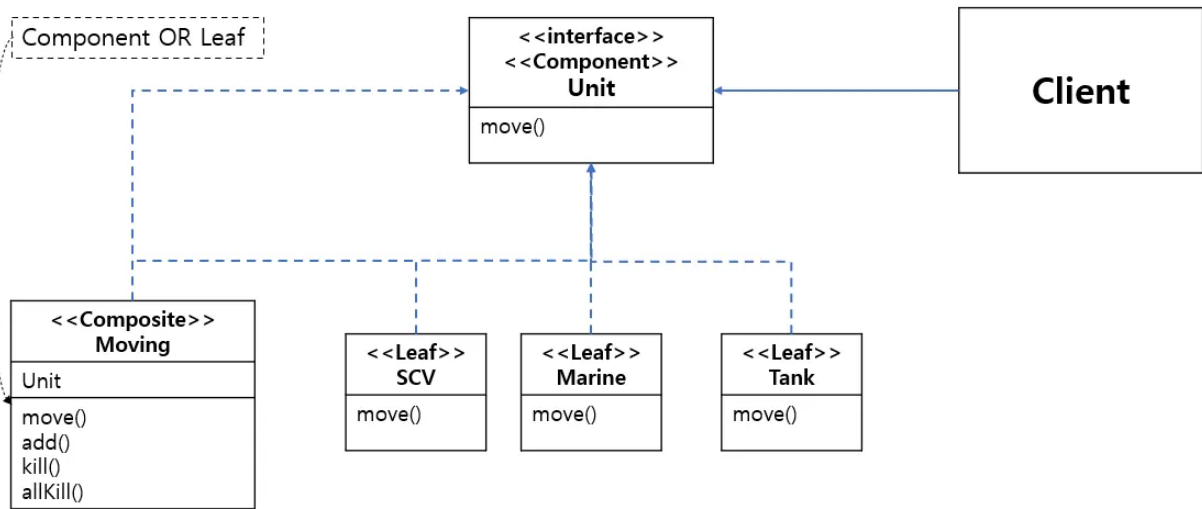
        for(Unit unit : units){
            unit.move();
        }
    }
}

////출력/////
마린 이동
SCV 이동
탱크 이동

테스트 코드에서 새로만든 units배열출력
마린 이동
SCV 이동
탱크 이동
탱크 이동
SCV 이동

```


- 예제 코드 UML



- 장점

1. 클라이언트 코드가 단순해진다 (다형성)

- 클라이언트는 복잡한 트리 구조를 몰라도 됩니다. 최상위 노드 하나만 잡고 명령을 내리면, 재귀적으로 알아서 자식들에게 전파됩니다.
- 예시:** `folder.getSize()`를 호출하면, 폴더 안에 있는 모든 파일의 용량을 합산해서 리턴합니다.

2. 새로운 종류의 객체 추가가 쉽다 (OCP 준수)

- 새로운 타입의 파일(예: `.mp4`)이나 새로운 타입의 폴더(예: `ZipFolder`)가 추가되어도, 클라이언트 코드는 수정할 필요 없이 기존 인터페이스를 그대로 사용하면 됩니다.

- 단점

1. 설계가 너무 일반화될 수 있다 (타입 안전성 문제)

- 모든 것을 동일하게 다루려다 보니, 말이 안 되는 상황이 생길 수 있습니다.
- 예시:** `File` 클래스에도 `add(Child)` 메소드가 있어야 인터페이스가 통일됩니다. 하지만 파일 안에 파일을 넣을 수는 없죠? 결국 `File.add()`를 호출하면 에러를 뱉거나 아무 일도 안 하게 처리해야 하는데, 이는 런타임에 혼란을 줄 수 있습니다.

🧑‍🎨 데코레이터 패턴

기본 기능을 가진 객체가 있고, 그 위에 부가적인 기능을 계속 덧붙이고 싶을 때가 있습니다. 상속을 쓰자니 경우의 수가 너무 많아질 때, **데코레이터 패턴**이 해결사가 됩니다.

"객체에 추가적인 요건(기능)을 동적으로 첨가하며, 기능 확장이 필요할 때 서브클래싱(상속) 대신 쓸 수 있는 유연한 대안 패턴"

■ 언제 사용하나요? (Why?)

1. 상속(Inheritance)으로 해결하려니 클래스가 폭발할 때

- **상황:** 커피 주문 시스템을 만듭니다. 에스프레소, 우유 추가, 시럽 추가, 휘핑크림 추가 기능이 필요합니다.
- **상속의 문제:** 상속을 쓰면 EspressoWithMilk, EspressoWithSyrup, EspressoWithMilkAndSyrup... 수십 개의 조합 클래스를 다 만들어야 합니다.
- **해결:** 데코레이터를 쓰면 Milk, Syrup 클래스만 만들고, 런타임에 new Milk(new Syrup(new Espresso()))처럼 조립하면 끝납니다.

2. 런타임에 객체의 기능을 동적으로 추가/삭제하고 싶을 때

- 프로그램 실행 중에 특정 객체에만 로그 기능을 붙이거나, 암호화 기능을 붙였다 뗄 수 있습니다.
- 예제코드

```
//컴포넌트 정의// - 수행할 서비스 정의
public interface CakeOption{
    String add();
}

// 기본 클래스 정의// - 가장 기본적인 서비스 정의
public class Cake implements CakeOption{

    @Override
    public String add(){
        return "케이크에 추가한 것 : ";
    }
}

//데코레이터 정의//
public abstract class Decorator implements CakeOption{
    CakeOption cakeoption;

    public Decorator(CakeOption cakeoption){
        this.cakeoption = cakeoption;
    }

    @Override
    public String add(){
        return CakeOption.add();
    }
}

//데코레이터를 상속해 구체 클래스 정의// - 추가 서비스 정의
public class Candle extends Decorator{
    public Candle(CakeOption cakeoption){
        super(cakeoption);
    }

    @Override
    public String add(){
        return super.add() + "초, ";
    }
}
```

```
}

public class Sign extends Decorator{
    public Sign(CakeOption cakeoption){
        super(cakeoption);
    }

    @Override
    public String add(){
        return super.add() + "글자데코,";
    }
}

public class Flower extends Decorator{
    public Flower(CakeOption cakeoption){
        super(cakeoption);
    }

    @Override
    public String add(){
        return super.add() + "꽃, ";
    }
}

/////테스트 코드///// -클라이언트
public class Client{
    private static boolean addCandle = true;
    private static boolean addSign = true;
    private static boolean addFlower = true;

    public static void main(String[] args){
        CakeOption cake = new Cake();

        if(addCandle){
            cake = new Candle(cake);
        }

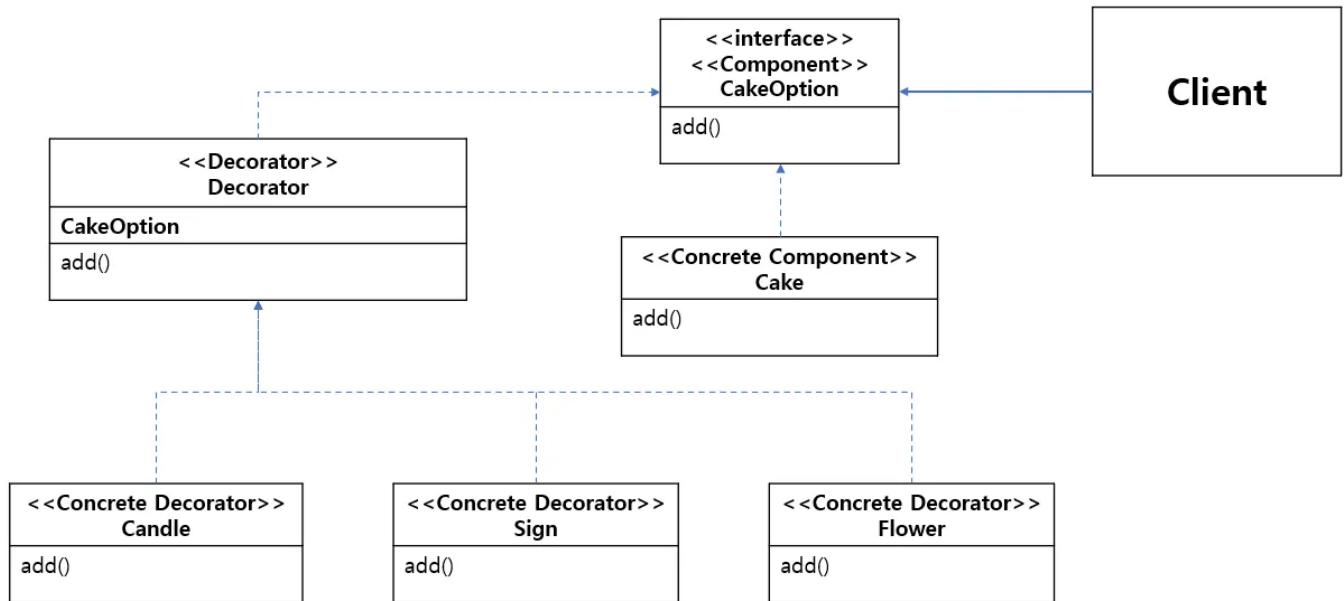
        if(addSign){
            cake = new Sign(cake);
        }

        if(addFlower){
            cake = new Flower(cake);
        }

        System.out.println(cake.add());
    }
}

/////출력/////
케이크에 추가한 것 : 초, 글자데코, 꽃,
```

- 예제 코드 UML



- 장점: 무한한 유연성

1. 상속보다 훨씬 유연하다 (조합의 마법)

- 상속은 컴파일 시점에 기능이 고정되지만, 데코레이터는 실행 시점(Runtime)에 마음대로 기능을 조합할 수 있습니다. 레고 블록처럼 필요한 기능만 쏙쏙 끼워 넣을 수 있죠.

2. 단일 책임 원칙(SRP) 준수

- 핵심 기능(예: 커피 내리기)과 부가 기능(예: 우유 타기)을 별도의 클래스로 분리하여 관리할 수 있습니다.

- 단점: 코드의 복잡성

1. 자잘한 객체들이 너무 많아진다

- 기능을 잘게 쪼개다 보니 작은 클래스들이 많아지고, 이를 조립하는 코드가 다소 지저분해 보일 수 있습니다.
- 예시:** `new BufferedReader(new InputStreamReader(new FileInputStream(...)))` ← 자바 하다가 한 번쯤 보셨죠? 이게 바로 데코레이터 패턴의 흔적입니다.

2. 디버깅이 어렵다

- 객체가 겹겹이 싸여 있어서, 에러가 났을 때 어느 겹질(Decorator)에서 문제가 발생했는지 추적하기가 까다로울 수 있습니다.

🏠 퍼사드 패턴(Facade Pattern) ★

집에 홈시어터를 설치했다고 상상해 봅시다. 영화 한 편을 보려면 TV 켜고, 셋톱박스 켜고, 스피커 켜고, 입력 모드 바꾸고... 리모컨 4개를 돌려가며 눌러야 합니다. 너무 복잡하죠? 이때 "영화 보기" 버튼 하나만 누르면 이 모든 과정이 자동으로 실행되게 만든다면 어떨까요? 이것이 바로 퍼사드 패턴입니다.

"복잡한 서브 시스템(Subsystem)들을 인터페이스 뒤에 숨기고, 사용자가 시스템을 쉽게 사용할 수 있도록 통합된 '단순한 인터페이스'를 제공하는 패턴"

'Facade(퍼사드)'는 건물의 정면(외관)을 뜻합니다. 건물 안의 복잡한 배관이나 전선은 벽 뒤에 숨기고, 사람들은 깔끔한 외관과 출입구만 이용하는 것과 같습니다.

■ 언제 사용하나요? (Why?)

1. 복잡한 시스템을 단순하게 쓰고 싶을 때

- 여러 개의 클래스를 순서대로 호출해야 하나의 기능이 완성되는 경우, 이를 감싸서 *****버튼 하나*****로 만듭니다.

2. 클라이언트와 서브 시스템 간의 의존성을 줄이고 싶을 때

- 클라이언트 코드가 서브 클래스 A, B, C, D...를 모두 알면 결합도가 높아집니다. 중간에 퍼사드를 두면 클라이언트는 퍼사드 하나만 알면 됩니다.
- 예제코드

```
// 복잡한 서브 클래스 //
public class Grinder(){
    public void grind(){
        System.out.println("커피 그라인딩");
    }
}

public class Water(){
    public void water(){
        System.out.println("물 넣기");
    }
}

public class Cream(){
    public void cream(){
        System.out.println("크림 넣기");
    }
}

public class Sugar(){
    public void sugar(){
        System.out.println("설탕 넣기");
    }
}

// 퍼사드 클래스 정의 //
public class CoffeeMachine{

    Grinder g = new Grinder();
    Water w = new Water();
    Cream c = new Cream();
    Sugar s = new Sugar();
}
```

```

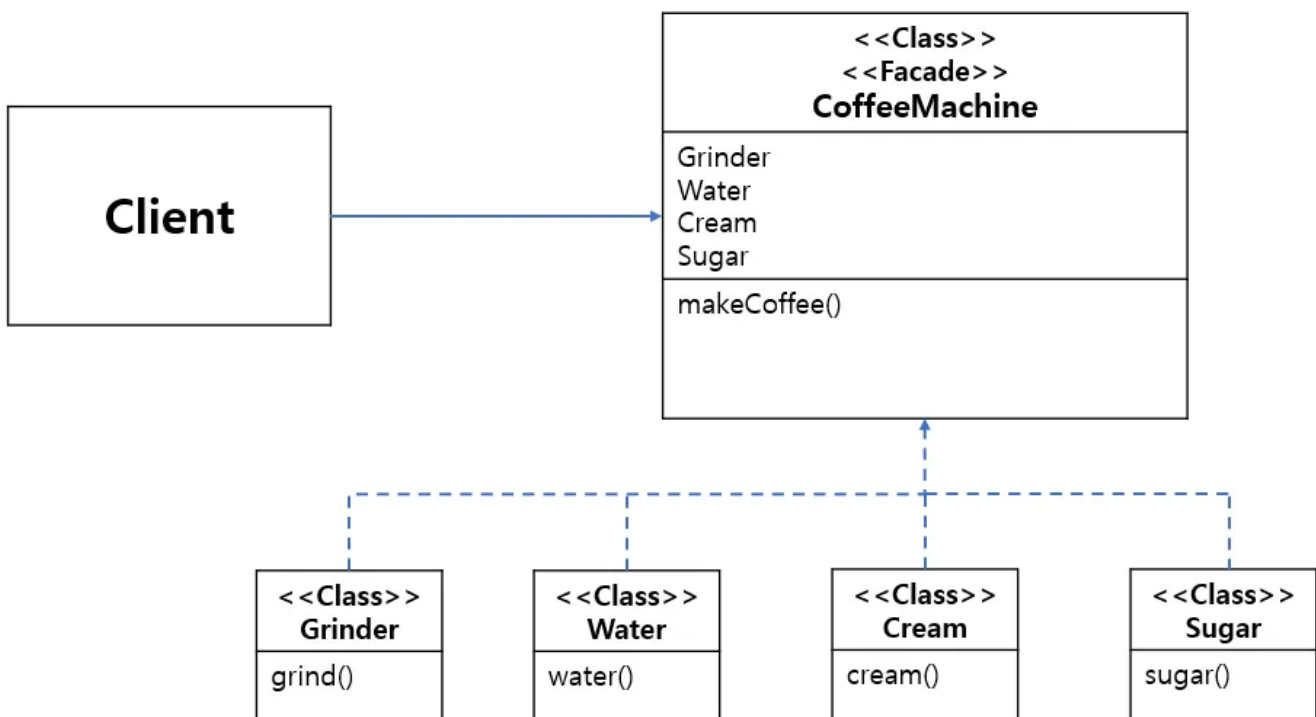
    public void makeCoffee(){
        g.grind();
        w.water();
        c.cream();
        s.sugar();
    }
}

// 클라이언트(테스트 코드) //
public class FacadePatternTest{
    public static void main(String[] args){
        // 퍼사드가 없을 때//
        /*
        Grinder g = new Grinder();
        Water w = new Water();
        Cream c = new Cream();
        Sugar s = new Sugar();
        g.grind();
        w.water();
        c.cream();
        s.sugar();
        */

        // 퍼사드 패턴 사용 //
        CoffeeMachine machine = new CoffeeMachine();
        machine.makeCoffee();
    }
}

```

- 예제 코드 UML



- **장점:** 단순함과 결합도 감소

1. 사용하기 쉽다 (가독성 향상)

- 클라이언트는 복잡한 내부 로직을 몰라도 됩니다. `facade.start()` 메소드 하나만 호출하면 되므로 코드가 매우 깔끔해집니다.

2. 결합도를 낮춘다 (Loose Coupling)

- 나중에 서브 시스템의 내부 코드가 바뀌어도, 퍼사드 코드만 수정하면 됩니다. 클라이언트 코드는 건드릴 필요가 없습니다.
- **단점:** 퍼사드 자체가 너무 비대해진다

1. 퍼사드가 '신(God)'이 될 수 있다

- 모든 서브 시스템의 의존성이 퍼사드 클래스 한 곳으로 몰리게 됩니다.
- 관리를 잘못하면 퍼사드 클래스 혼자서 수천 줄이 넘어가고, 모든 클래스를 참조하는 `***'갓 오브젝트 (God Object)'`가 되어 유지보수가 힘들어질 수 있습니다.
- **서브 시스템:** 청소부, 요리사, 짐꾼, 주차 요원 (손님이 이들을 일일이 찾아다니면 힘들겠죠?)
- **퍼사드(Facade):** 프론트 데스크 직원
- **클라이언트(손님):** 프론트 직원에게 "룸서비스 주세요"라고 말만 하면, 직원이 알아서 요리사에게 주문 넣고 짐꾼을 시켜 배달해 줍니다. 손님은 요리사의 전화번호를 몰라도 됩니다.
- 스프링에서 Service객체의 존재 이유

컨트롤러는 복잡한 로직을 몰라야 합니다. "**Service 객체의 메소드 하나만 호출하면, 그 안에서 Repository도 부르고, 외부 API도 부르고, 알림도 보낸다**"는 구조 자체가 퍼사드 패턴의 철학입니다.

🕒 플라이웨이트 패턴(Flyweight Pattern)

객체를 `new`로 무한정 만들어내면 언젠가 메모리(RAM)가 터져버립니다(OOM). 똑같은 데이터를 가진 객체라면 굳이 여러 개 만들 필요가 있을까요? 하나만 만들어서 돌려쓰면 어떨까요? 이것이 바로 **플라이웨이트 패턴**입니다.

"많은 수의 유사한 객체들을 매번 생성하지 않고, 변하지 않는 속성(Intrinsic)을 가진 객체를 공유(Sharing)하여 메모리 사용량을 줄이는 패턴"

이름 그대로 'Flyweight(가벼운 체급)'처럼 객체를 가볍게 만드는 것이 목표입니다.

■ 언제 사용하나요? (Why?)

1. 엄청나게 많은 수의 유사한 객체를 생성해야 할 때

- **상황:** 게임에서 '나무'를 100만 그루 심어야 합니다. 나무 객체 하나가 1KB라면, 100만 개면 1GB 메모리가 필요합니다.
- **해결:** 나무의 '형태(3D 모델, 텍스처)'는 모두 똑같습니다. 이 형태 정보는 **딱 하나만 생성해서 공유**하고, 나무마다 다른 '위치(x, y)' 정보만 따로 관리하면 메모리를 획기적으로 줄일 수 있습니다.

2. 객체의 속성이 대부분 변하지 않을 때

- 자주 변하지 않는 속성(Intrinsic State)을 분리하여 재사용할 수 있을 때 사용합니다.
- 예제코드 - (출처 : <https://dev-youngjun.tistory.com/217>)

```
// 변하지 않는 속성을 지닌 공유객체//
public final class Font{ //공유하는 객체이므로 final로 선언
    final String family;

    final int size;

    public Font(String Family, int size){
        this.family = family;
        this.size = size;
    }

    public String getFamily() {
        return family;
    }

    public int getSize() {
        return size;
    }
}

// 변하는 속성을 지닌 객체//
public class Character{
    private char value;

    private String color;

    private Font font; //변하지 않는 속성

    public Character(char value, String color, Font font){
        this.value = value;
        this.color = color;
        this.font = font;
    }
}

public class FontFactory{
    private Map<String, Font> cache = new HashMap<>();

    public Font getFont(String font){
        if(cache.containsKey(font)) { //이미 존재하는 폰트이면
            return cache.get(font); // 기존의 인스턴스 반환
        } else{ //없으면 새로만들어 저장
            String[] split = font.split(":");

            Font newFont = new Font(split[0], Integer.parseInt(split[1]));
            cache.put(font, newFont);
            return newFont;
        }
    }
}
```



```

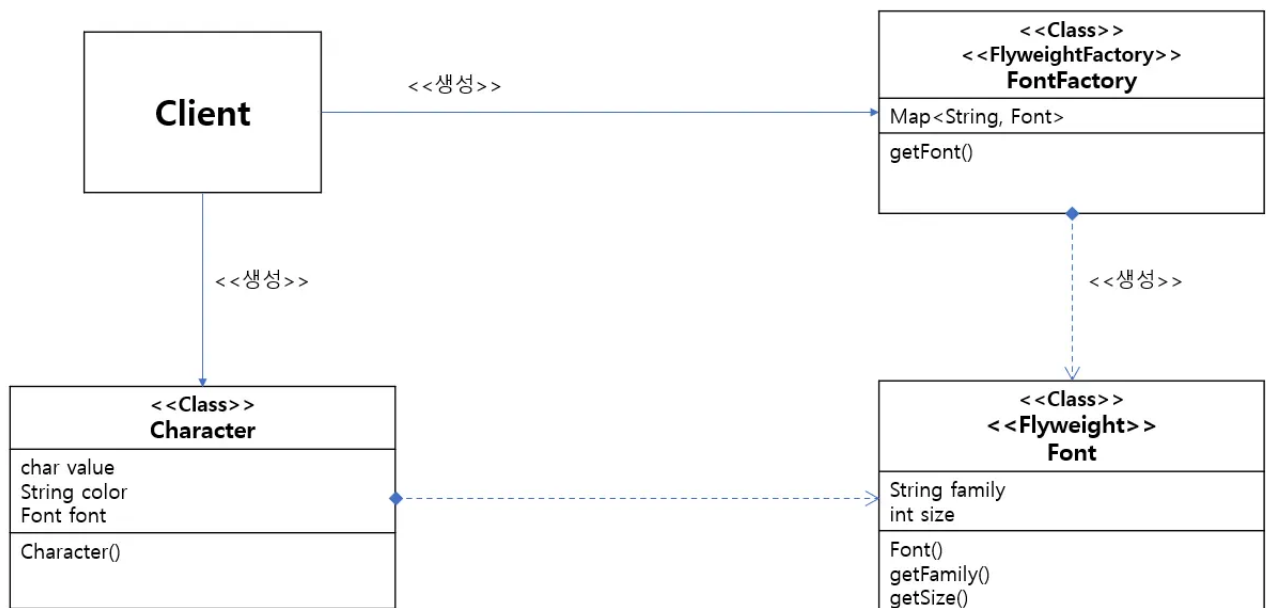
    }
}

public class FlyWeightPatternTest{
    public static void main(String[] args){
        FontFactory fontFactory = new FontFactory();

        // nanum:12가 없으니 Font추가
        Character c1 = new Character('h', "white",
fontFactory.getFont("nanum:12"));
        // nanum:12가 존재하므로 기존에 만든 객체 반환
        Character c2 = new Character('e', "white",
fontFactory.getFont("nanum:12"));
        // nanum:12가 존재하므로 기존에 만든 객체 반환
        Character c3 = new Character('l', "white",
fontFactory.getFont("nanum:12"));
    }
}

```

- 예제코드 UML



- 장점: 메모리 다이어트

1. 메모리 사용량을 획기적으로 줄인다

- 중복되는 데이터를 공유하므로, 객체가 아무리 많아져도 실제 메모리 점유율은 크게 늘어나지 않습니다.

2. 생성 속도가 빨라진다

- 이미 만들어진 객체 풀(Pool)에서 가져다 쓰기 때문에, 매번 `new`를 호출하는 오버헤드가 줄어듭니다.

- 단점: 코드의 복잡성

1. 객체의 값을 함부로 바꿀 수 없다 (공유의 부작용)

- 공유 객체의 값을 하나라도 바꾸면, 그 객체를 공유해서 쓰는 **모든 곳에서 값이 바뀌어 버립니다**. 따라서 공유되는 속성은 반드시 ****불변(Immutable)****이어야 합니다.

2. 코드가 복잡해진다

- 객체의 데이터를 ****'공유되는 것(Intrinsic)'**과 ****'공유되지 않는 것(Extrinsic)'**으로 나누어 설계해야 하므로 구현 난이도가 올라갑니다.

"자바에서 String a = "Hello";와 String b = "Hello";를 만들면, 자바는 내부적으로 메모리에 "Hello"를 딱 하나만 만들고 a와 b가 이를 같이 가리키게 합니다. 이것이 바로 플라이웨이트 패턴이 적용된 대표적인 사례입니다. (반면 new String("Hello")는 강제로 새 객체를 만듭니다.)"

프록시 패턴(Proxy Pattern) ★

중요한 사람(CEO)을 만나려면 바로 집무실로 들어갈 수 없습니다. 반드시 **비서실**을 거쳐야 하죠. 비서는 약속된 사람인지 확인(보안)하거나, CEO가 바쁘면 잠시 기다리게(지연 로딩) 할 수 있습니다. 이 '비서' 역할을 하는 것이 바로 **프록시 패턴**입니다.

"어떤 객체에 직접 접근하는 것이 아니라, 해당 객체를 대리(Proxy)하는 객체를 통해 대상 객체에 접근하는 방식을 사용하는 패턴"

'Proxy'는 대리인이라는 뜻입니다. 클라이언트는 대리인(Proxy)에게 요청을 보내고, 대리인이 실제 객체(Real Subject)를 제어하거나 필요한 시점에 생성해서 요청을 전달합니다.

■ 언제 사용하나요? (Why?)

1. 객체 생성을 최대한 미루고 싶을 때 (가상 프록시, Lazy Initialization)

- 상황:** 용량이 1GB나 되는 고해상도 이미지를 로딩해야 하는데, 사용자가 스크롤을 내리기 전까지는 굳이 메모리에 올릴 필요가 없습니다.
- 해결:** 프록시 객체(겉데기)만 먼저 보여주고, 실제 이미지는 ****'진짜 필요한 시점'*****에 생성해서 메모리에 올립니다.

2. 접근 권한을 제어하고 싶을 때 (보호 프록시)

- 상황:** 중요한 인사 정보 데이터는 관리자만 볼 수 있어야 합니다.
- 해결:** 프록시가 클라이언트의 권한을 먼저 검사하고, 권한이 있는 경우에만 실제 데이터 객체에 접근을 허용합니다.

3. 부가 기능을 추가하고 싶을 때 (로깅 프록시)

- 기존 코드를 건드리지 않고, 메소드 실행 시간을 측정하거나 로그를 남기는 기능을 앞에 덧붙일 수 있습니다. (OCP 준수)
- 예제코드

```
public interface Act{
    public void act();
}

//실제 클래스 생성//
```

```

public class RealClass implements Act{

    @Override
    public void act(){
        System.out.println("실제 클래스 실행!");
    }
}

//프록시 클래스 생성//
public class Proxy implements Act{
    RealClass r = new RealClass(); //실제 클래스 생성

    @Override
    public void act(){
        //접근 권한
        r.act(); //실제 클래스 실행
        //log
    }
}

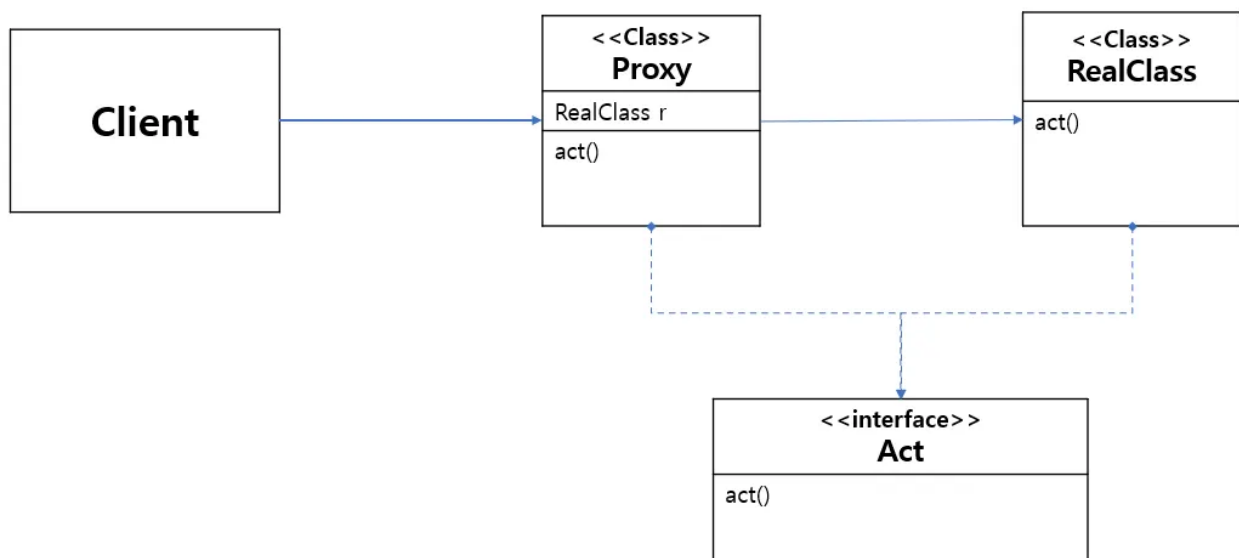
//클라이언트(테스트코드)//
public class ProxyPatternTest{
    public static void main(String[] args){
        Proxy p = new Proxy();

        p.act(); //프록시 클래스의 act실행
    }
}

//출력//
실제 클래스 실행!

```

- 예제코드 UML



- 장점: 제어와 확장

1. 기존 코드를 수정하지 않고 새로운 기능을 추가할 수 있다 (OCP)

- 실제 객체는 자기 할 일(비즈니스 로직)만 하고, 권한 체크나 로깅 같은 부가 작업은 프록시가 담당하므로 역할이 깔끔하게 분리됩니다. (SRP 준수)

2. 성능 최적화가 가능하다 (캐싱)

- 프록시가 결과를 저장(Caching)해 두었다가, 똑같은 요청이 오면 실제 객체까지 가지 않고 바로 응답을 줄 수 있습니다.
- 단점: 복잡도 증가

1. 코드가 복잡해진다

- 단순히 객체를 바로 쓰면 되는데, 중간에 프록시 클래스를 하나 더 만들어야 하므로 코드 양이 늘어납니다.

2. 반응 속도가 느려질 수 있다

- 객체 생성 시점은 늦출 수 있지만, 매번 프록시를 거쳐야 하므로 아주 미세한 성능 저하가 발생할 수 있습니다.

"여러분이 스프링에서 자주 쓰는 @Transactional 어노테이션이 바로 프록시 패턴으로 동작합니다. 여러분이 만든 클래스 앞단에 스프링이 몰래 '트랜잭션 시작/종료'를 담당하는 프록시를 만들어 끼워 넣기 때문에, 코드 한 줄 없이도 트랜잭션 관리가 되는 마법이 일어나는 것입니다."